Aplicación funcionando · demo local

CONTROL DOCUMENTAL · ASISTENTE CON CITAS

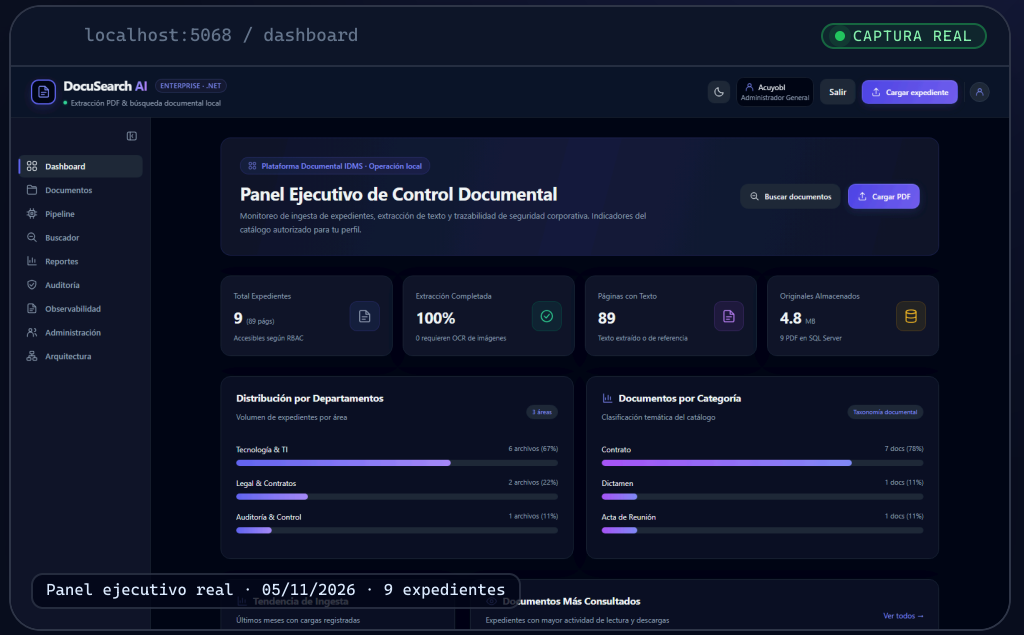
Tus documentos, encontrados y auditables.

Control documental local con búsqueda inteligente, asistente que cita sus fuentes y auditoría completa. La IA es opcional; el control, no.

Blazor Interactive Server | ASP.NET Core 10 | EF Core 10 | SQL Server Express | OpenCode 2 | DeepSeek

AF Allan Flores Valverde
HA Huberth Aguilar Fallas
CH Carlos Hernández Molina

[COMPLETAR: institución / evento] • 2026 • 10 diapositivas · 11 minutos



10 módulos funcionales

87 pruebas sin servicios

102 con SQL Server y OpenCode

PROBLEMA Y PROPUESTA

El documento correcto no debería ser un acto de fe.

El conocimiento vive en PDF, pero encontrarlo, protegerlo y demostrar quién lo consultó sigue siendo manual.

Dispersión

Carpetas, correos y discos personales. Encontrar el contrato correcto cuesta más que leerlo.

Riesgo de exposición

Subir PDF confidenciales a servicios externos no es aceptable para la organización.

Sin trazabilidad

Sin permisos ni bitácora, cada búsqueda es una incógnita y cada descarga una fuga potencial.

ANTES

Buscar en carpetas y confiar en la memoria de quien estuvo.

→

DESPUÉS

Preguntar, leer la cita y auditar la consulta en el mismo sistema.

DocuSearch AI existe para que **encontrar, restringir y demostrar** sean la misma operación.

ARQUITECTURA · CÓMO FUNCIONA

Cuatro capas, una sola máquina.

Interfaz, reglas de negocio, datos y una capa de IA opcional. Los permisos y la validación viven en C#.

1 Experiencia

Blazor Interactive Server con Razor y CSS. El JavaScript propio solo corre en el navegador.

Dashboard · Catálogo · Visor · Asistente

→

2 Dominio y seguridad · C#

Autenticación TOTP, RBAC, ingesta, auditoría y orquestación del asistente.

Auth TOTP · RBAC · DocumentService · Auditoría

→

3 Datos

EF Core 10 sobre SQL Server Express: metadatos, páginas, originales y trazas.

Metadatos · Páginas JSON · Originales · Trazas

4 · IA opcional

Nunca decide permisos. Solo recibe fragmentos autorizados.

OpenCode · servidor local por HTTP, herramientas desactivadas

LangChain · worker Python sobre OpenCode 2

Extractivo · reglas deterministas siempre disponibles

- ✓ RBAC y validación de citas en C#, no en el modelo.
- ✓ Originales en SQL Server con SHA-256; integridad al servir.
- ✓ Todo local: sin nube obligatoria y con la IA desactivable.

CÓMO FUNCIONA · FLUJO PRINCIPAL

Del PDF a la respuesta citada, sin pasos opacos.

Seis pasos con validación en cada frontera: límites antes de guardar, hash al servir y permisos al consultar.

1. **Límites y hash**
   Formato, 25 MB, 500 páginas y SHA-256 antes de tocar la base.
2. **Texto por página**
   PdfPig extrae la capa de texto; sin texto se marca «Requiere OCR».
3. **Resumen y entidades**
   Resumen extractivo y entidades por reglas, sin inventar contenido.
4. **Guardado transaccional**
   Metadatos y original en SQL Server dentro de una transacción.
5. **Búsqueda autorizada**
   Relevancia por términos y filtros; el RBAC se aplica en servidor.
6. **Respuesta con cita**
   El asistente cita páginas concretas y el servidor valida la cita.

25 MB por PDF · **500 páginas** máximo · **20 PDF** por lote · **SHA-256** recalculado al servir · **Transacción** por archivo

EVIDENCIA REAL · ENDPOINT DE SALUD

```
# Respuesta real del sistema en local
GET /health  →  200
{
  "status":    "ready",
  "database":  "SQL Server",
  "assistant": "langchain+opencode:deepseek/deepseek-flash"
}
```

La app arranca con o sin motor; si el motor falla, responde en modo extractivo.

DÓNDE ESTÁ EL CONTROL

- ☑ Límites y formato validados antes de escribir en la base.
- ☑ Hash SHA-256 comparado en cada apertura o descarga.
- ☑ Permisos comprobados al buscar, abrir, descargar y responder.

TEMA CENTRAL · ASISTENTE CON CITAS

El modelo propone; el sistema verifica.

La recuperación, el permiso y la validación de citas están en C#. El modelo solo recibe fragmentos autorizados.

01
Consulta
Lo que escribe el usuario en el buscador.

→

02
Fuentes autorizadas
Máx. 6 fragmentos × 1000 caracteres, filtrados por rol.

→

03
Modelo sin herramientas
No ve archivos, SQL ni internet.

→

04
Citas en C#
Cada cita debe existir en las fuentes enviadas.

→

05
Con fuente
Texto y cita verificable; si falla, extractivo.

QUÉ NUNCA RECIBE EL MODELO

- El PDF completo o la base de datos.
- El historial de conversación.
- Documentos fuera de tus permisos.
- Herramientas para ejecutar acciones.

QUÉ VALIDA EL SERVIDOR

- Que cada cita exista en las fuentes enviadas.
- Permiso al recuperar y permiso al responder.
- Respuesta JSON con **answer** y **citations**.
- Fallback extractivo etiquetado si el motor falla.

Nota técnica: el modelo no decide permisos ni qué documentos existen; recibe un contexto acotado y sus citas se validan contra ese mismo contexto. Si el motor falla, la respuesta pasa a modo extractivo etiquetado, sin inventar una fuente.

EL AGENTE · MOTORES Y MODELOS

OpenCode, LangChain y extractivo.

El catálogo se descubre en el OpenCode de cada equipo; LangChain orquesta la generación en Python y el motor se cambia desde el asistente sin reiniciar.

LOCAL · HTTP

OpenCode

Servidor local con las herramientas desactivadas en cada consulta. Catálogo de modelos gratis y de pago según los proveedores conectados.

Sin herramientas · Gratis / pago

ACTIVO · PYTHON

LangChain + OpenCode

Orquesta plantilla, cadena y salida estructurada en Python; el modelo se invoca en OpenCode 2. No requiere Ollama.

Pydantic · stdin / stdout

SIEMPRE DISPONIBLE

Extractivo

Reglas deterministas para listas, fichas, entidades y páginas exactas. No inventa contenido.

Sin LLM · Fallback

CATÁLOGO DINÁMICO · SIN MODELOS FIJOS

GRATIS	Modelos con coste 0 de proveedores conectados; pueden agotar su cuota, pero siguen visibles en el selector.
PAGO	DeepSeek Flash y Pro del proveedor conectado: reciben únicamente las fuentes autorizadas.
LOCALES	Ollama, LM Studio o llama.cpp: opcionales y ocultos por defecto.

CONTROL EN LA CABECERA DEL ASISTENTE

- El administrador general cambia de motor sin editar archivos ni reiniciar.
- «Actualizar motores» valida la conexión y recarga el catálogo sin enviar documentos.
- La cabecera muestra el motor activo y su estado («Verificado» o «sin conexión»).
- Si la cuota gratuita se agota, se muestra el error del proveedor y se ofrece revisión humana.

Sin modelos hardcodeados: el catálogo depende del OpenCode de cada equipo. Si no hay ningún motor disponible, la app sigue respondiendo en modo extractivo por reglas.

SEGURIDAD Y CALIDAD

La confianza se demuestra con controles y evidencia.

Autenticación en dos pasos, permisos en servidor, integridad verificable y una bitácora que sobrevive al reinicio.

Acceso TOTP

RFC 6238: código de 6 dígitos, 30 s, un solo uso y tolerancia ±1.

RBAC en servidor

Rol y departamento se comprueban en cada consulta, visor y descarga.

Acceso local

Loopback y sin cabeceras de reenvío; la demo Docker lo desactiva por el NAT del contenedor.

Integridad SHA-256

El hash se recalcula al servir; una discrepancia cancela la entrega.

QUIÉN PUEDE VER QUÉ · RBAC

ROL	LECTURA	CARGA / EDICIÓN	ADMINISTRACIÓN
Administrador general	Todos	Sí	Sí
Admin. departamental	Hasta Confidencial · su área	Sí	Parcial
Gestor documental	Hasta Confidencial	Sí	No
Consultor	Público y Uso interno	No	No
Auditor	Hasta Estrictamente Secreto	No	Solo auditoría

Evidencia: si el hash no coincide al servir un PDF, la entrega se cancela y se registra `INTEGRITY_MISMATCH` con estado `ADVERTENCIA`; la bitácora y las trazas del agente persisten en SQL Server.

RESULTADOS Y CALIDAD

Lo que ya funciona, medido.

Cifras tomadas del repositorio y de la suite de comprobaciones, no de una diapositiva de marketing.

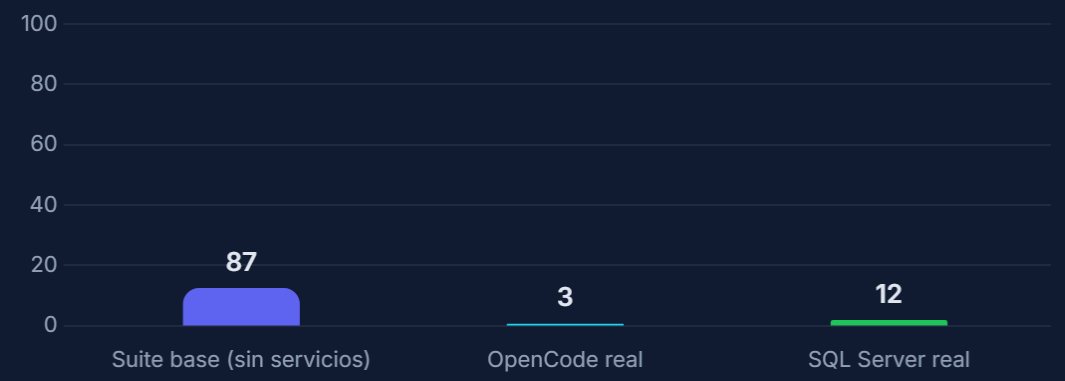
87
pruebas sin servicios
datos, permisos, PDF reales, TOTP y límites

102
con base real
incluye OpenCode real y SQL Server aislado

10
módulos entregados
rutas autenticadas y flujos cubiertos por pruebas

CI
en cada cambio
build + pruebas en GitHub Actions

SUITE DE COMPROBACIONES · 102 EN TOTAL



QUÉ YA ESTÁ ENTREGADO

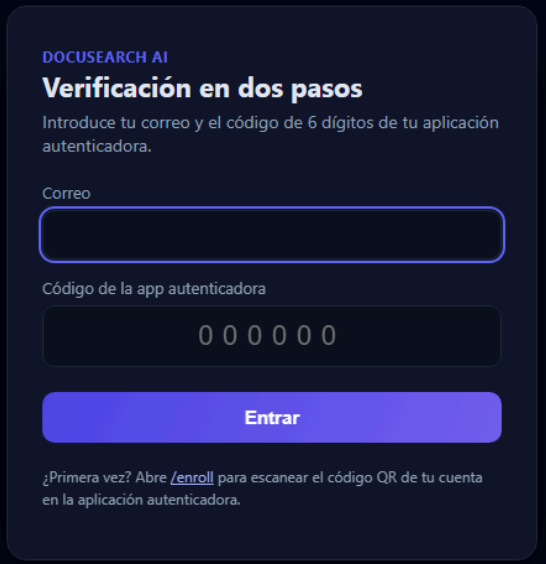
- 10 módulos: dashboard, catálogo, carga, pipeline, buscador, visor, reportes, auditoría, observabilidad y administración.
- Límites reales: 25 MB, 500 páginas y 20 PDF por lote.
- Migraciones idempotentes y base de pruebas separada `DocuSearch_tests`.
- Datos reales: 9 expedientes, 89 páginas y más de 170 movimientos registrados.

Sin métricas inventadas: las cifras salen de la documentación y de la suite del proyecto; el gráfico desglosa el total de 102.

DEMO REAL · CAPTURAS DE LA APP EN EJECUCIÓN

El App real, paso a paso.

67 segundos por 13 capturas reales: 9 expedientes, 89 páginas, 6 departamentos y 3 usuarios de esta instalación.



01 **Acceso TOTP**
Correo autorizado + código temporal de 6 dígitos.

3 usuarios

EQUIPO Y CIERRE

Un equipo, una promesa: cada respuesta con su fuente.

Producto, desarrollo y pruebas en un mismo proyecto: DocuSearch AI.

EQUIPO DEL PROYECTO

Allan Flores Valverde
DocuSearch AI · 2026

Huberth Aguilar Fallas
DocuSearch AI · 2026

Carlos Hernández Molina
DocuSearch AI · 2026

Blazor · ASP.NET Core 10 · SQL Server · OpenCode 2 · DeepSeek

APRENDIZAJES

- La IA suma cuando el contexto y los permisos son del sistema, no del modelo.
- Local-first es viable: sin nube obligatoria y con integridad demostrable.
- Decir lo que falta — OCR y búsqueda semántica — genera más confianza que prometerlo.

PRÓXIMOS PASOS

- Identity u OIDC con TOTP/passkeys y TLS para publicar.
- OCR para escaneos y búsqueda híbrida con embeddings.
- Almacenamiento externo y mayor capacidad que SQL Express.

Menos carpetas. Más decisiones.

Gracias · ¿Preguntas? · DocuSearch AI · 2026

`demo local · http://localhost:5068`